Университет ИТМО

Мегафакультет компьютерных технологий и управления

Факультет программной инженерии и компьютерной техники

Отчет
по лабораторной работе №3
«сервис-ориентированная архитектура»

Вариант 67226

Выполнили:

Кортыш Константин

Байрамгулов Мунир

Преподаватель

Цопа Евгений Алексеевич

Санкт-Петербург, 2026 г.

Задание

Переработать веб-сервисы из лабораторной работы №2 таким образом, чтобы они реализовывали основные концепции микросервисной архитектуры. Для этого внести в оба сервиса — «вызываемый» и «вызывающий» — перечисленные ниже изменения.

Изменения в «вызываемом» сервисе:

- Переработать сервис, обеспечив возможность его развёртывания на платформе Spring Cloud.
- Использовать все основные компоненты архитектуры Spring Cloud: Config Service, Load Balancer, Service Discovery.
- В качестве Service Discovery использовать Eureka.
- В качестве балансировщика нагрузки использовать Ribbon.
- Все запросы к сервисам должны проксироваться через API Gateway. В качестве API Gateway использовать Zuul Proxy.

Изменения в «вызывающем» сервисе:

- Разделить приложение на два модуля — веб-приложение с веб-сервисом и EJB-jar с бизнес-компонентами.
- Переместить всю логику из класса сервиса в Stateless EJB. В классе сервиса оставить только обращение к методам бизнес-интерфейса. EJB-компонент должен быть доступен удалённо (иметь Remote-интерфейс).
- Сформировать на уровне сервера приложений пул компонентов EJB настраиваемой мощности, динамически расширяемый при увеличении нагрузки.
- Настроить второй экземпляр сервера приложений на другом порту, «поднять» на нём вторую копию веб-сервиса и пула EJB.
- Настроить балансировку нагрузки на оба запущенных узла через Haproxy.

Оба веб-сервиса и клиентское приложение должны сохранить полную совместимость с API, реализованными в рамках предыдущих лабораторных работ.

Выполнение

Исходный код

https://github.com/msuny-c/soa

Каталог	Содержимое
`services/worker-service/`	Worker Collection Service: Spring Boot 2.3, Spring Cloud Hoxton, Eureka-клиент
`services/config-server/`	Spring Cloud Config Service
`services/eureka-server/`	Service Discovery на Eureka
`services/api-gateway/`	API Gateway на Zuul Proxy с балансировщиком Ribbon
`services/hr-service/`	HR Service: модули `hr-api`, `hr-ejb`, `hr-war`, `hr-ear`
`config-repo/`	конфигурации сервисов для Config Service
`services/site/`	клиент: React, Ant Design, TanStack Query — без изменений
`docker/`, `compose.yaml`	запуск всего стека в Docker

Диаграмма развёртывания

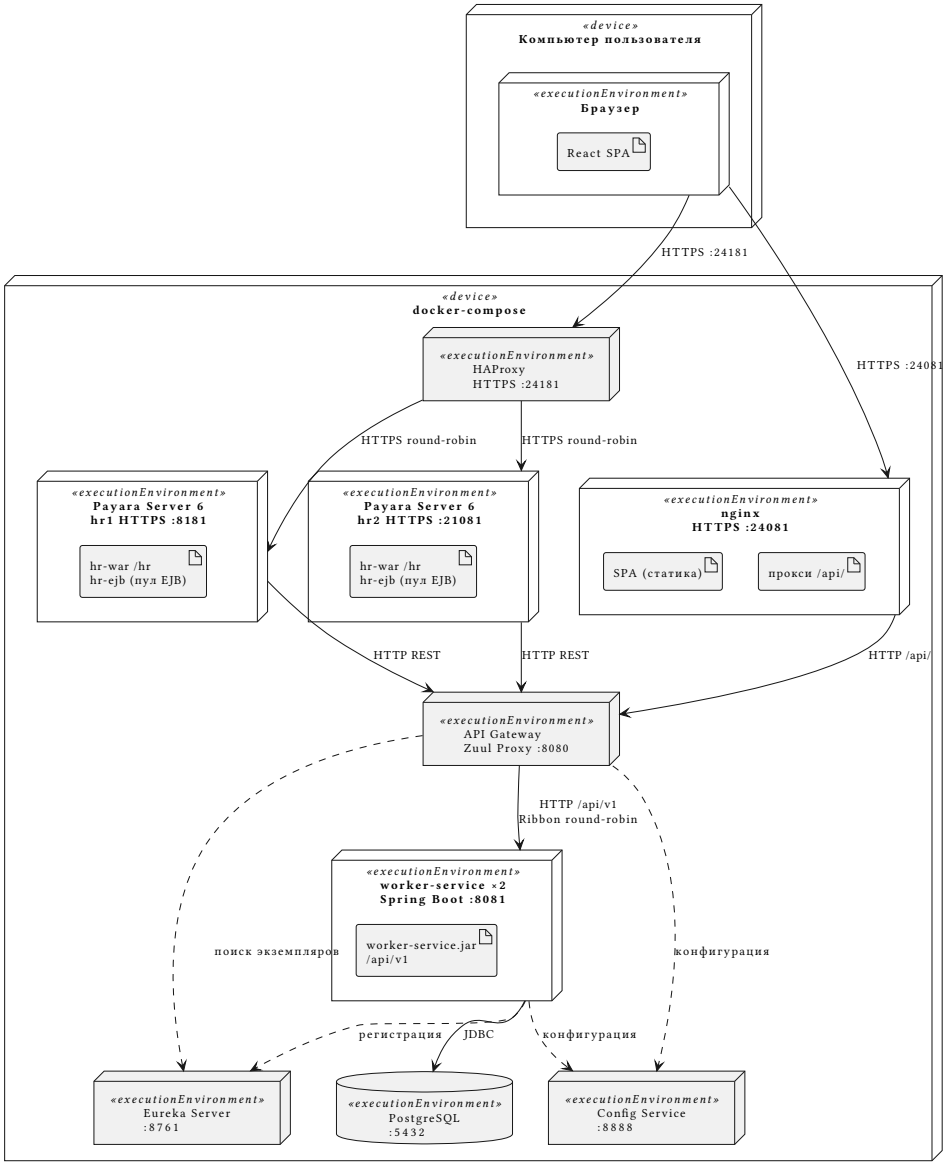


Рис. 1. Диаграмма развёртывания

Worker Collection Service (Spring Cloud)

«Вызываемый» сервис переработан с Spring MVC-приложения под Payara на самостоятельное Spring Boot-приложение (Spring Boot 2.3, Spring Cloud Hoxton, Java 11) и запускается в двух экземплярах. REST API полностью унаследован из ЛР1 (`/api/v1/workers` и сопутствующие операции), изменён только способ разворачивания.

- Конфигурация (подключение к PostgreSQL, параметры экземпляра) выдаётся при старте Config Service из репозитория `config-repo`.
- Каждый экземпляр регистрируется в Eureka под именем `worker-service`.

Инфраструктура Spring Cloud

Компонент	Порт	Назначение
Config Service	`8888`	централизованная конфигурация сервисов (native-репозиторий `config-repo`)
Eureka Server	`8761`	Service Discovery: регистрация и поиск экземпляров
API Gateway (Zuul Proxy)	`8080`	единая точка входа: маршрут `/api/v1/**` на `worker-service`

Маршрут проксируется через Ribbon, который по данным Eureka распределяет запросы между двумя экземплярами `worker-service` по алгоритму round-robin; при выходе экземпляра из строя Eureka исключает его из списка, и трафик автоматически идёт на оставшийся.

HR Service (Stateless EJB)

Приложение разделено на модули, собираемые в EAR:

Модуль	Содержимое
`hr-api`	Remote-бизнес-интерфейс `IndexationRemote` и Serializable-DTO
`hr-ejb`	Stateless EJB `IndexationBean`: вся бизнес-логика индексации зарплат
`hr-war`	веб-сервис JAX-RS: только валидация входа и вызовы бизнес-интерфейса
`hr-ear`	сборка EAR для развёртывания на Payara

Пул EJB настроен в дескрипторе `glassfish-ejb-jar.xml`: стартовый размер 4, шаг расширения 2, максимум 32. На первых запросах контейнер заполняет пул до стартового размера, под нагрузкой пул динамически расширяется, не занятые экземпляры возвращаются к стартовому размеру по таймауту.

Приложение развёрнуто на двух экземплярах Payara: `hr1` на порту 8181 и `hr2` на порту 21081; на каждом работает своя копия веб-сервиса и пула EJB.

Балансировка HR Service (HAProxy)

HAProxy терминирует HTTPS на порту 24181 и распределяет запросы между `hr1` и `hr2` (round-robin) с проверкой доступности узлов; недоступный узел исключается из балансировки. Доступна страница статистики на порту 24183. Вызовы Worker Collection Service оба экземпляра HR Service выполняют через API Gateway, поэтому также проходят через балансировку Ribbon.

Клиентское приложение

Клиент (React, Ant Design) не изменён: внешние адреса сервисов сохранены как в ЛР2. Статика SPA раздаётся тем же входом, что и API (nginx перед API Gateway на порту 24081), вызовы HR Service идут через HAProxy на порту 24181.

HTTPS

- Внешние входы системы — nginx (порт 24081) и HAProxy (порт 24181) — работают только по HTTPS с самоподписанными сертификатами; сертификаты генерируются при сборке образов (`keytool` + `openssl`).
- Экземпляры Payara обслуживают только HTTPS (`http-listener-1` отключён).
- Внутренние вызовы между контейнерами (gateway–worker, hr–gateway) выполняются по HTTP внутри изолированной сети docker.

Выводы

«Вызываемый» сервис переведён на платформу Spring Cloud: конфигурация выдаётся Config Service, экземпляры регистрируются в Eureka, все запросы проксируются через Zuul API Gateway и балансируются Ribbon между двумя экземплярами. «Вызывающий» сервис разделён на EJB-jar и WAR: бизнес-логика перенесена в Stateless EJB с Remote-интерфейсом и настраиваемым пулом, приложение развёрнуто на двух экземплярах Payara за HAProxy. API обоих сервисов и клиентское приложение полностью совместимы с ЛР1–ЛР2.